

COMS W4156 Advanced Software Engineering (ASE)

October 15, 2024

On this day ...



#MeToo Movement

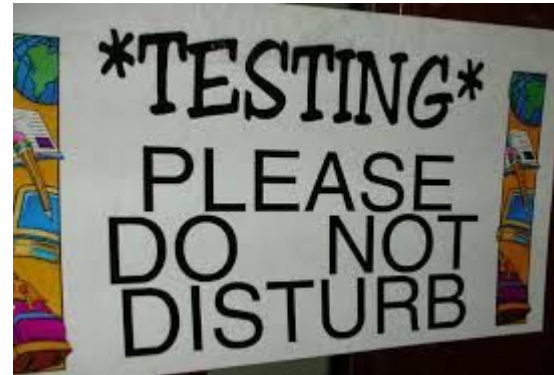
2017 Actress [Alyssa Milano](#)'s tweet "If you've been sexually harassed or assaulted write 'me too'" prompts flood of replies across Facebook, Instagram and Twitter



Actress and Activist
[Alyssa Milano](#)

Agenda

1. **Class Testing**
2. Integration Testing
3. Integration Testing demo: Nikita Rajaneesh
4. Reminders

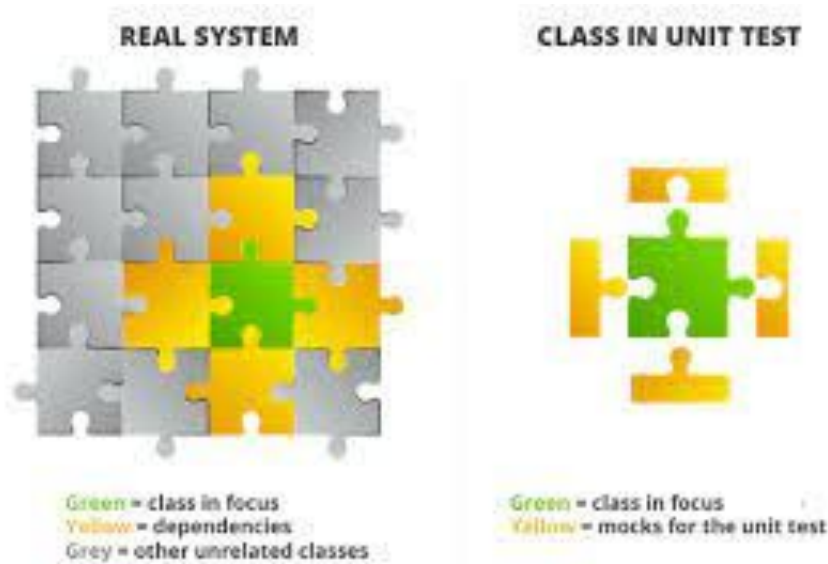


Class Testing

Class (module) testing tests that individual classes do what they are supposed to do, often conducted together with [unit testing](#) or considered part of unit testing

During unit testing of the units in a class, i.e., functions or methods, first mock all code outside each unit, then repeat unit tests using other code from the same class while still mocking all code outside the class

Only redo those unit tests where the unit indeed calls other units in same class!



Repeating Unit Tests for Class Testing

```
myClass foo (myClass obj, something x) {  
    local anotherThing y;  
  
    some code;  
    y = methodInSameClass(obj,x);  
    some more code;  
    obj.field = methodOutsideClass(y);  
    code that does something;  
  
    return obj;  
}
```



During initial unit testing of foo, we should fake both method calls to return known values

Then during class testing, we should call the real methodInSameClass and use its real output, but fake the call to methodOutsideClass to return some known value(s)

Later during integration testing across classes, we call the real methodOutsideClass

Choosing Inputs for Class Testing

Equivalence partitions + boundary conditions technique for choosing inputs for unit testing may, or may not, conveniently exercise all dependencies between units in the same class

Another notion of "coverage" = exercising all direct and up to Nth degree indirect (transitive) dependencies between units in the same class (typically only up to $N=2$)



Coverage for Class Testing

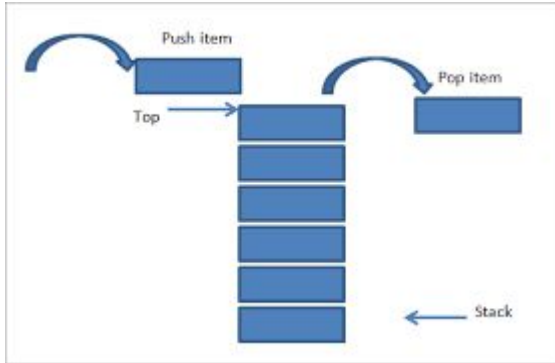
To achieve good coverage of dependencies, leverage conventional branch coverage in a special case of whitebox testing: Peek inside the units to determine the execution paths whereby one unit calls another unit, and choose inputs to each calling unit that force it to call every possible callee unit

Work backwards from a call to `methodInSameClass` through any branches and loops to the beginning of the caller `foo`, to determine the [path conditions](#) (constraints) on `foo`'s own inputs needed to force execution to reach that call to `methodInSameClass`

```
myClass foo (myClass obj, something x) {  
    local anotherThing y;  
  
    some code that may change x;  
    if (something rarely true about x) {  
        y = methodInSameClass(obj,x);  
    }  
    some more code that may change x;  
    if (something often true about x) {  
        obj.field = methodOutsideClass(y);  
    }  
    code that does something;  
  
    return obj;  
}
```



State Transition Testing



Tracing execution paths to force calls between units in the same class does not address cases where two or more units share data but neither calls the other - e.g., `push()` and `pop()` are dependent on each other because they *share* the stack data structure, but they do not normally call each other

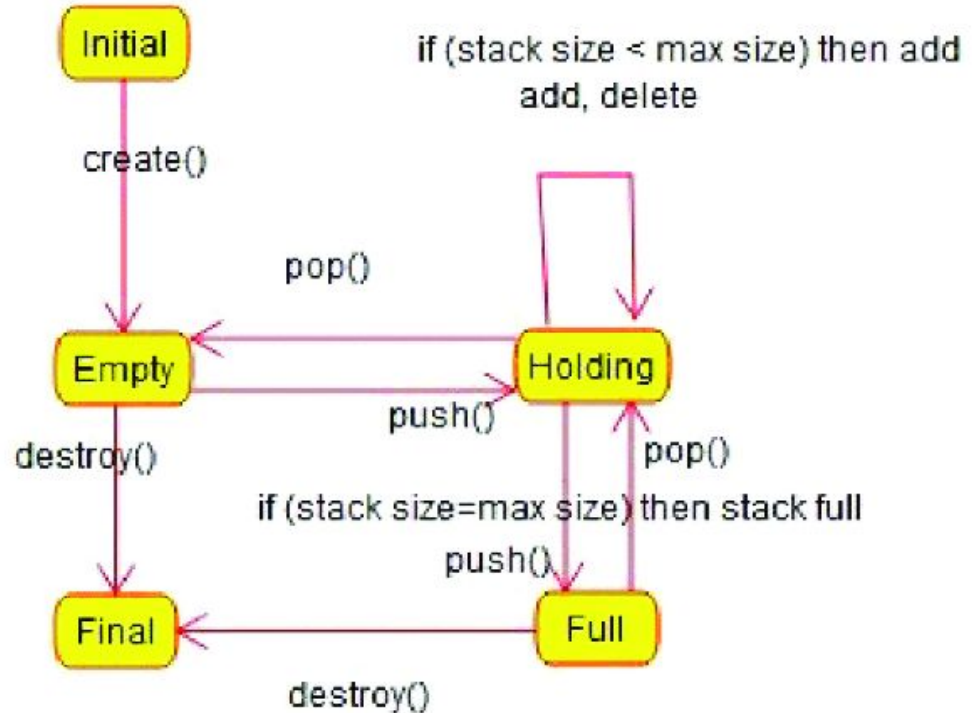
State transition testing assumes state shared among the functions of a class, so may not apply to “utility” classes that group functions that share little if any state

Example: Testing a Stack Data Structure

Draw “[state diagram](#)” (finite state automaton) to visualize the possible states and their transitions

The idea is to choose combinations of methods and inputs to those methods that exercise all the transitions between states

There may be constraints on some transitions



Example: Testing a Stack Data Structure

Test 1: create()

Test 2: destroy()

Test 3: create() - need new one

Test 4: push(xxx)

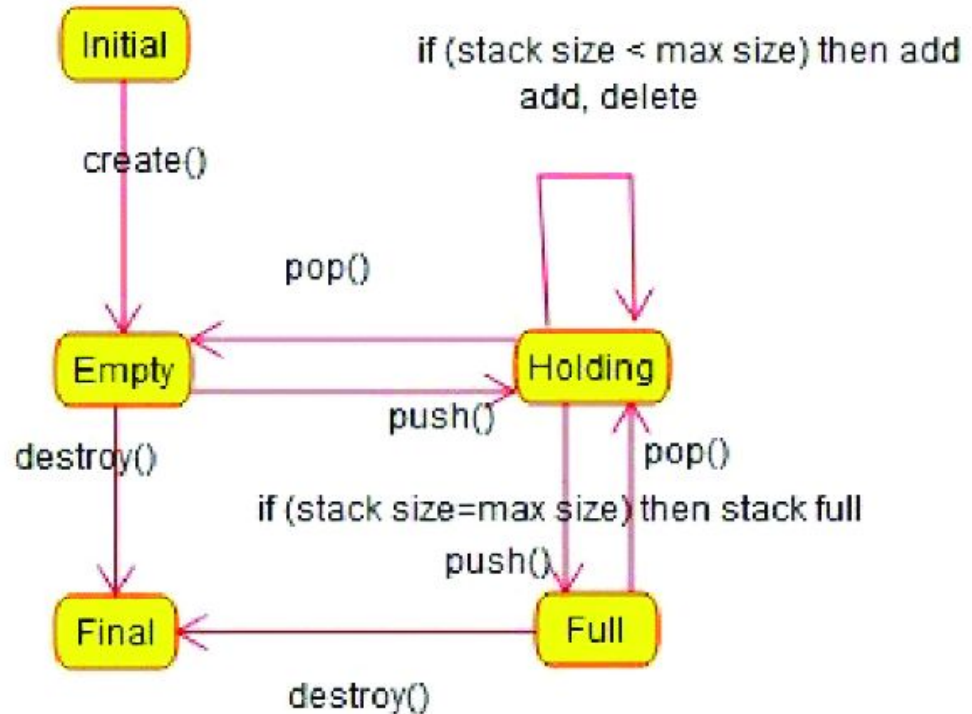
Test 5: pop() - did we get xxx?

Test 6: push(yyy)

Test 7: push(zzz)

Test 8: pop() - did we get zzz?

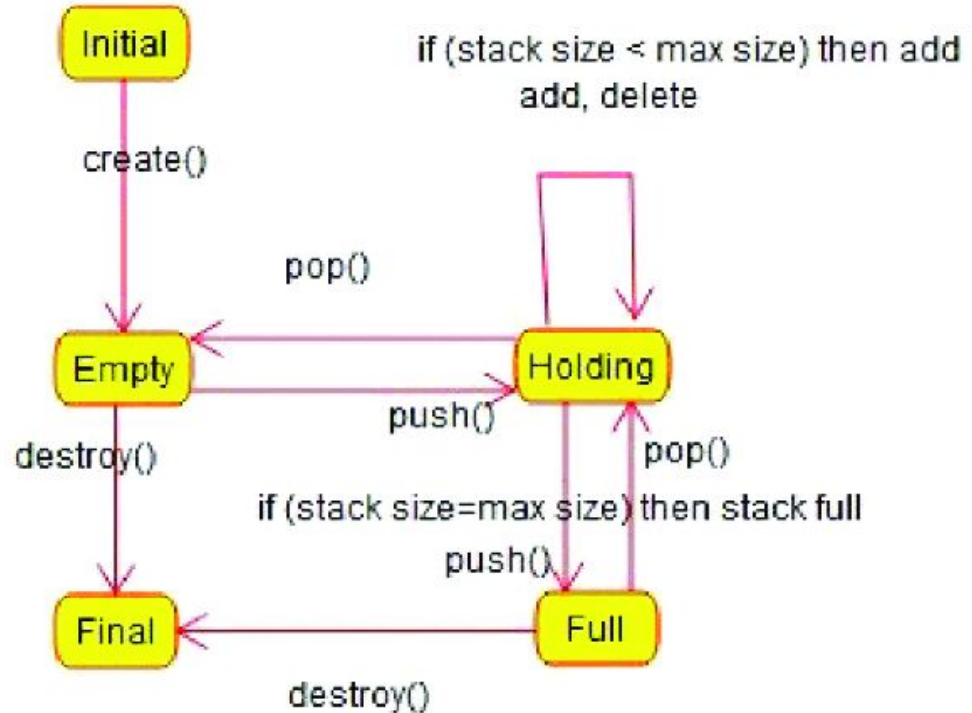
And so on...



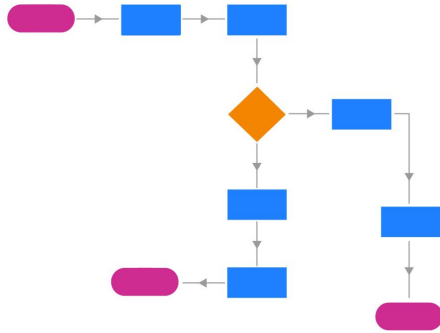
Example: Testing a Stack Data Structure

This could go on forever, need “stopping condition”

- Follow each full path without loops
- Do each loop zero, one and two times around
- Also try *invalid* paths - method sequences disallowed by constraints or not included in diagram



State Transition Testing Also Applies to (Some) API Testing



For many APIs, the appropriate ordering among API calls follows state transitions within the API implementation. Does not work for APIs consisting of utility functions that share little or no state

API developers should test using combinations of entry points and inputs to those entry points that exercise all possible transitions between states

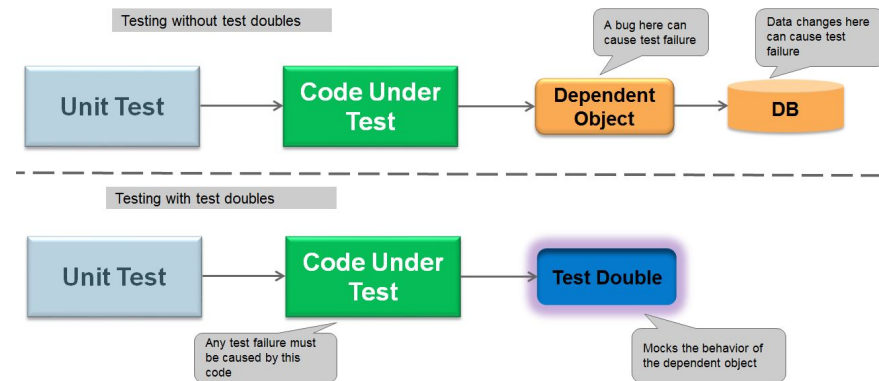
API developers should document the state transition diagram so client developers can test their client code's dependencies on the API by exercising the transitions between states

Testing in Isolation

Unit testing aims to test each unit and module *in isolation from rest of program*

The unit, or a series of units in the same class, is called from a test runner or another kind of test driver, not from the other code in the rest of the program

When the units under test would normally call other code in the rest of the program outside the class, or otherwise interact with the environment outside the class, that other code and/or environment is faked



Exceptions: basic libraries like math and string processing (just assume it works)



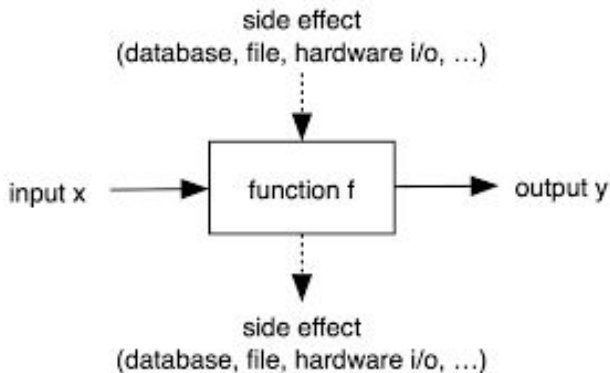
Class Testing with Side-Effects Outside the Class

Avoid when possible! But if your methods have, or might have, side-effects outside the class (i.e., beyond the object's own fields and the class' static fields), then unit testing needs to consider what to do with these outputs that change global shared state and/or produce externally visible output

Need to fake that global shared state and/or external visible output so it isn't really global and it isn't really externally visible

Unit tests need to be very fast, so they can run whenever the unit is changed.

Real resources often too slow. And bugs found during unit testing should be in the unit under test, not in some other code.



Agenda

1. Class Testing
2. Integration Testing
3. Integration Testing demo: Nikita Rajaneesh
4. Reminders

$$Life = \int_{birth}^{death} \frac{happiness}{time} \Delta time$$

What is Integration Testing?

The goal of [integration testing](#) is to find errors at the *interfaces* between components (classes or other modular units) and assemblies consisting of multiple components

Considers two or more components that communicate or depend on each other in some way (or where one component communicates with or depends on another component, does not need to be reciprocal)

Often automated via the same testing tools as unit testing, with test runners, fixtures that setup and teardown constructed inputs, assertions that check outputs

	Unit Testing	Integration Testing
		
		



Integration Testing with Test Doubles

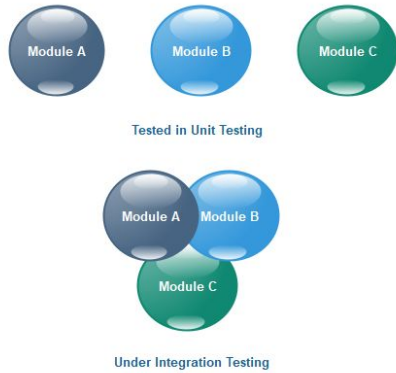
Like unit and class testing, integration testing should fake dependencies outside the code under test

When an integration test fails, we want to know the bug is in one or more of the components being integrated into the assembly, or at the interface being integrated, not some other part of the system

However, integration testing eventually needs to test interfaces with external resources - in which case real resources are used after initial testing with test doubles

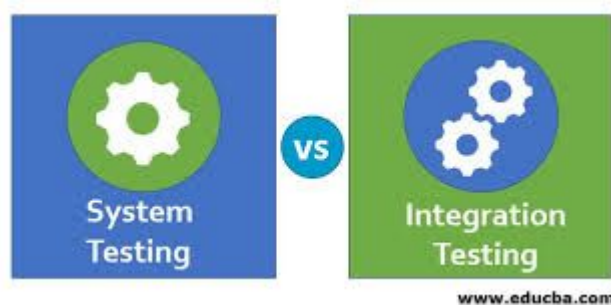


Choosing Integration Testing Inputs



Similar to class testing except 1. some integration order strategy is usually chosen in advance and 2. source code may not be available for all components to be integrated (e.g., third-party libraries and services)

Given components (or previously integrated assemblies of components): reuse their unit/class testing inputs (e.g., from equivalence partitioning + boundary analysis), replace test doubles with real code from the other components being integrated, choose additional test inputs to force reaching replaced test doubles as needed



Example

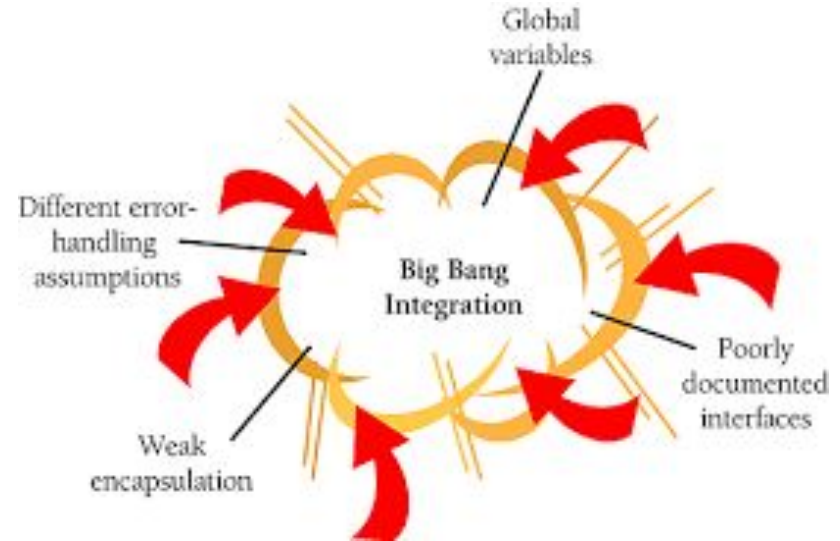
```
myClass foo (myClass obj, something x) {  
    local anotherThing y;  
  
    some code that may change x;  
    if (something rarely true about x) {  
        y = methodInSameClass(obj,x);  
    }  
    some more code that may change x;  
    if (something often true about x) {  
        obj.field = methodOutsideClass(y);  
    }  
    code that does something;  
  
    return obj;  
}
```

Work backwards from a call to methodOutsideClass through any branches and loops to the beginning of the caller foo, to determine the *path conditions* (constraints) on foo's own inputs needed to force execution to reach that call to methodOutsideClass



Integration Order Strategies

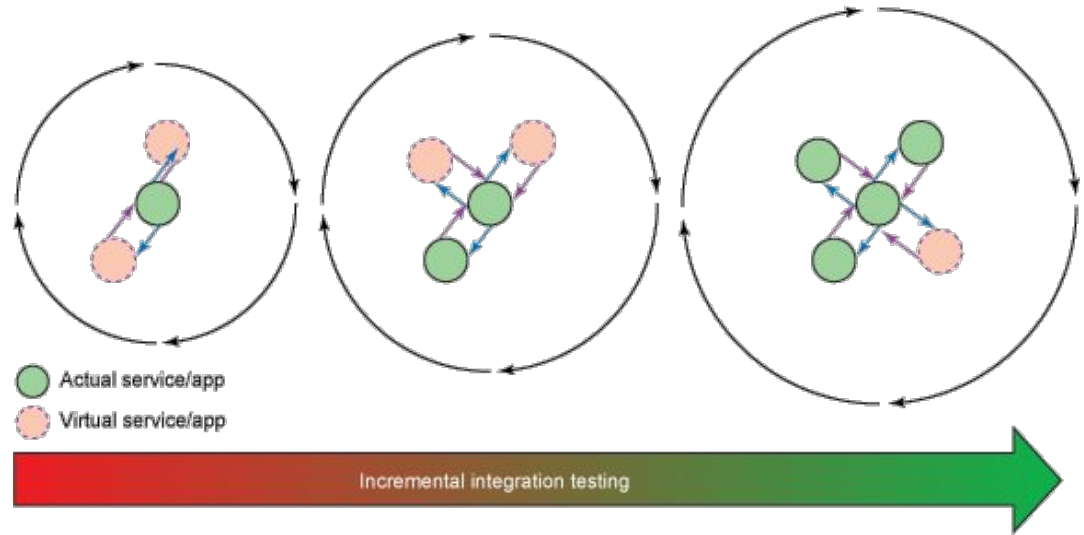
“Big bang” approach to integration testing: Test all independently developed components in isolation, then test the whole system. All done!



Not so fast: Bugs detected during “big bang” could be anywhere!

Incremental Integration Helps Localize Bugs

At each step, pick two components (or assemblies of components) that have both already been tested individually and at least one is dependent in some way on the other. Continue using test doubles for all other components



Since only a small number of components (ideally two) are integrated at a time, that limits where we need to look to find the “[root cause](#)” (the underlying coding mistake or interface mismatch) and fix the bug

Some Integration Testing Strategies

Risk: Prioritize testing the *most risky* interfaces, e.g., a new component being introduced to an existing system, replacing an existing third-party component with an alternative component from a different third party, replacing an existing third-party component with a new version of that component

History: Prioritize testing interfaces where the most bugs, or the most challenging bugs, have been found previously

Expense: Prioritize testing the interfaces where debugging is the most difficult and/or expensive



Agenda

1. Class Testing
2. Integration Testing
3. Integration Testing demo: Nikita Rajaneesh
4. Reminders



Nikita Rajaneesh Integration Testing Demo

[demo slides](#)



Agenda

1. Class Testing
2. Integration Testing
3. Integration Testing demo: Nikita Rajaneesh
4. Reminders



Deadlines

[Team First Iteration](#) due **this Friday** October 18

[Team First Iteration Demo](#) due the following Friday October 25

First Assessment

Friday October 25th 12:01am through Monday October 28th 11:59pm -

What to expect will be discussed Thursday



Thursday

What to expect on first assessment

If time permits, logging



Ask Me Anything

